

Projeto de Identificação de Sistemas e Controle PID

Guia para Estudantes de C213

Disciplina: Sistemas de Controle Automático

14 de abril de 2026

1 Sistemas de Supervisão e Interface Homem-Máquina

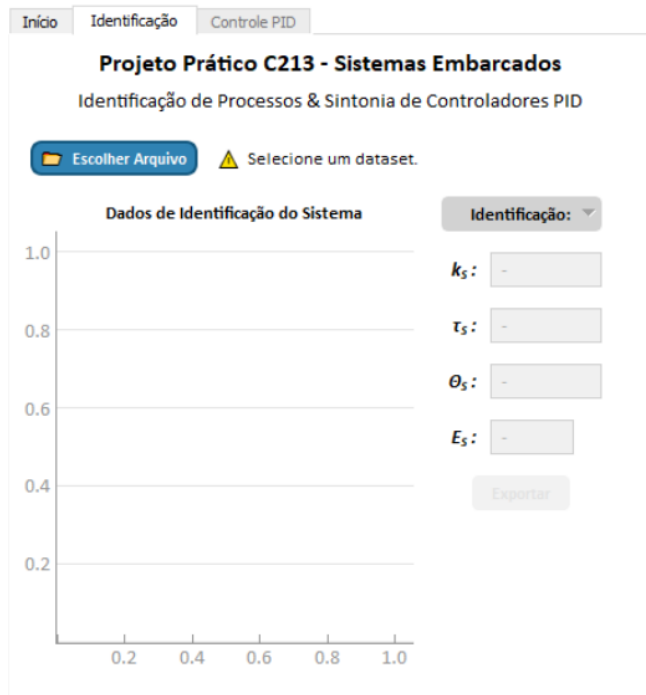
Sistemas de supervisão e interfaces homem-máquina (IHM) são ferramentas essenciais para o monitoramento e controle em tempo real de processos industriais. A IHM permite aos operadores visualizar dados, configurar setpoints e analisar métricas de desempenho, como tempo de subida, *overshoot* e erro em regime permanente. No contexto deste projeto, a IHM deve ser desenvolvida em Python (usando PyQt5 e Qt Designer) ou MATLAB e incluir:

A IHM multi-abas das Figuras 1a e 1b apresenta uma interface de exemplo para sistemas com Controle PID, permitindo diversas interações do usuário. Na segunda tela, é possível selecionar um conjunto de dados base para o processo aplicado, realizar a identificação pelos Métodos de Smith e Sundaresan e verificar os resultados. Na terceira tela, há duas seleções para definir a forma de Sintonia do Controlador PID.

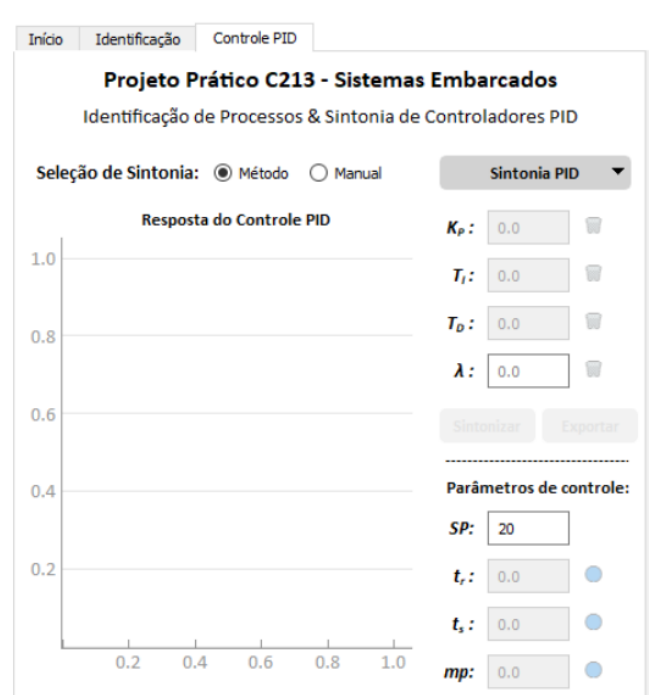
- O conjunto de dados selecionado em 1a é verificado como válido e apresenta o resultado para o Método de Identificação com menor EQM. É possível selecionar os dados de qualquer método em uma lista suspensa. A seleção de dados libera o acesso à aba 11b;
- Em 1b, a opção *Método* permite que o usuário selecione um entre os Métodos Clássicos de Sintonia. Nessa situação, os parâmetros K_p , T_i e T_d são calculados e não podem ser editados. Apenas é permitida entrada para o parâmetro λ caso seja selecionado o Método IMC;
- Em 1b, se selecionada a opção *Manual*, os valores dos parâmetros K_p , T_i e T_d podem ser editados, enquanto a seleção de Métodos Clássicos e edição para λ são bloqueadas. Há ícones que permitem a limpeza dos valores digitados neste modo.

Há dois botões de ação no sistema. Se habilitada a opção *Manual*, o botão *Sintonizar* realiza o Controle PID com parâmetros manuais, previamente verificando a estabilidade do processo para o conjunto de valores. Após a sintonia, por qualquer forma válida, o botão *Exportar* salva o gráfico como uma imagem, permitindo acesso futuro aos dados.

Na seção *Controle*, um campo de entrada permite ao usuário definir o valor para o SetPoint considerando uma entrada do tipo degrau. O valor inicial para o SetPoint é o mesmo do conjunto de dados do processo analisado, selecionado em 11a. A seguir, demais campos parametrizam os pontos de interesse da curva de resposta: tempos de subida e acomodação e índice de overshoot. As interações do sistema permitem marcar pontos com descrição de dados no gráfico.



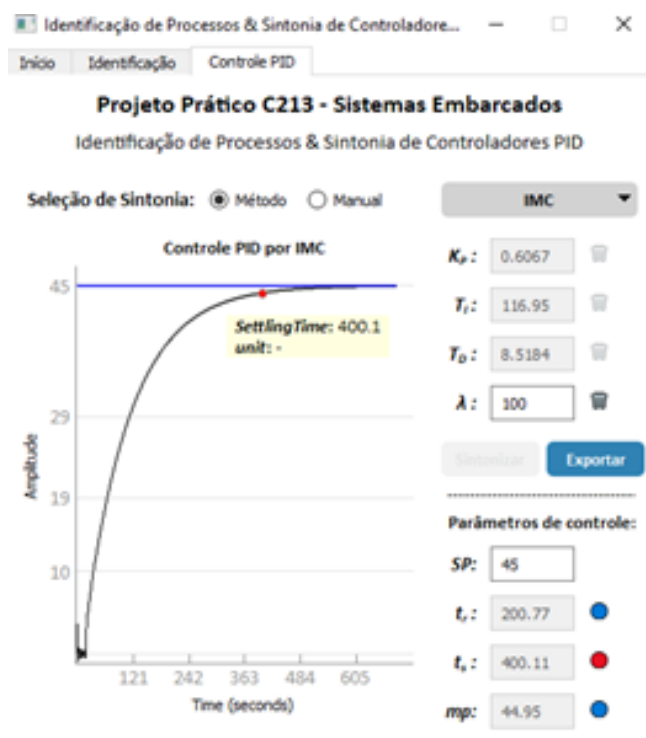
(a) Aba 'Identificação' para verificação de conjuntos de dados.



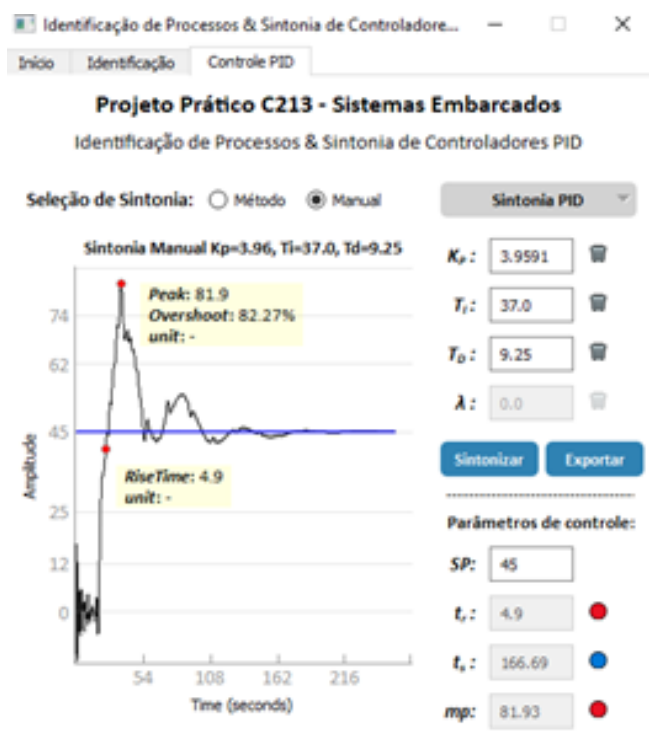
(b) Aba 'Controle PID' para parametrização do Controlador e métricas de resposta.

Figura 1: Descrição da aplicação da IHM para sistemas com Controle PID.

As Figuras 2a e 2b apresentam o funcionamento da interface para as duas opções de seleção da forma de Sintonia, por Métodos ou Manual, respectivamente, além da alteração do SetPoint e indicação dos pontos e valores de referência. Se o conjunto de dados possuir metadados de unidades de medida ou grandezas, as métricas são incluídas na figura e labels.



(a) Funcionamento da Interface para seleção de Métodos Clássicos de Sintonia.



(b) Funcionamento da Interface para seleção de parâmetros manuais.

Figura 2: Detalhamento das situações de aplicação da interface.

2 Contexto e Objetivos

O presente projeto tem como objetivo aplicar os fundamentos matemáticos e metodológicos no desenvolvimento de uma aplicação computacional que permita aos estudantes carregar um dataset experimental, identificar o modelo FOPDT da planta térmica, implementar diferentes técnicas de sintonia PID e visualizar o desempenho do sistema em malha aberta e malha fechada. Assim, busca-se integrar a teoria de controle clássico com uma aplicação prática e didática que facilite a compreensão dos efeitos de cada estratégia de sintonia em um processo real.

2.1 Objetivos do Projeto

Este projeto tem como finalidade que os estudantes desenvolvam uma aplicação completa de identificação de sistemas e projeto de controladores PID, integrando conhecimentos teóricos e habilidades práticas de programação. Ao completar este projeto, os estudantes serão capazes de:

- Carregamento de datasets experimentais (`.mat`).
- Identificação por Smith da curva em malha aberta e exibição dos parâmetros identificados (k , τ , θ).
- Projetar controladores PID usando métodos clássicos de sintonia
- Seleção de métodos de sintonização (Ziegler-Nichols, IMC, CHR, CC e ITAE) de acordo com excel dos grupos.
- Visualização de gráficos em tempo real, comparando respostas em malha aberta e fechada controlada.
- Painel de métricas: tempo de subida (t_r), tempo de acomodação (t_s), *overshoot* (M_p) e erro em regime permanente (malha aberta).
- Ajuste fino dos parâmetros PID
- Criação de uma interface gráfica
- Analisar respostas de sistemas de controle e suas características
- Visualizar dados e resultados de forma efetiva

3 Especificações do Projeto

3.1 Arquitetura da Aplicação

A aplicação deve seguir o padrão Modelo-Vista-Controlador (MVC) com os seguintes componentes:

1. Interface do Usuário: Três abas principais

- Aba de Identificação de Sistemas
- Aba de Controle PID
- Aba de Gráficos
- Aba de Login/Autenticação (Opcional)

2. Módulos de Processamento:

- Carregamento e processamento de dados

- Algoritmos de identificação
- Métodos de sintonia PID
- Simulação de respostas

3. Visualização:

- Gráficos interativos
- Legendas automáticas
- Marcadores de pontos importantes
- Exportação de figuras

3.2 Funcionalidades Requeridas

3.2.1 Aba de Identificação

- Carregamento de arquivos .mat com dados experimentais
- Visualização de dados de entrada e saída
- Implementação dos métodos Smith
- Cálculo automático do EQM (Erro Quadrático Médio)
- Exportação de gráficos

3.2.2 Aba de Controle PID

- Seleção entre sintonia automática e manual
- Implementação de múltiplos métodos de sintonia:
- Simulação de resposta do sistema controlado
- Cálculo automático de características de resposta
- Marcadores visuais em pontos importantes
- Legendas identificativas nos gráficos

Sintonize um controlador PID de acordo com os métodos especificados para seu grupo na Tabela 4. Em seguida, verifique o comportamento do sistema controlado por meio da resposta ao degrau unitário e analise o desempenho em termos de: Para a sintonia baseada no modelo de *Internal Model Control* (IMC), a escolha do parâmetro λ é livre e deve ser justificada de acordo com o desempenho desejado. Em geral:

- Valores pequenos de λ resultam em uma resposta mais rápida, mas com maior sensibilidade a ruídos e incertezas do modelo.
- Valores maiores de λ tornam a resposta mais lenta, mas garantem maior robustez e menor sobreleitura devido ao atraso.

Assim, a seleção de λ deve equilibrar a rapidez da resposta e a robustez do sistema controlado, devendo ser explicitamente justificada no relatório de cada grupo.

GRUPO	MÉTODO
1	CHR com sobresinal + CC
2	Ziegler-Nichols + ITAE
3	CHR + ITAE
4	Ziegler-Nichols + CC
5	IMC + ITAE
6	CHR + CC
7	IMC + ITAE
8	CHR com sobresinal + CC
9	Ziegler-Nichols + ITAE

Tabela 1: Distribuição dos grupos e métodos de sintonia PID.

3.3 Requisitos Técnicos

- **Linguagem:** Python 3.9 ou superior- MATLAB 2024Ra
- **Controle:** biblioteca python-control

4 Entregáveis

4.1 Código Fonte

- Código completo da aplicação.
- Arquivos de configuração e dependências.
- Datasets de teste.
- README.

4.2 Documentação

- Documentação técnica do código
- Explicação dos algoritmos implementados
- Análise dos resultados obtidos

4.3 Apresentação

- Demonstração ao vivo da aplicação
- Explicação das decisões de design
- Análise comparativa dos métodos
- Discussão sobre limitações e melhorias futuras

4.4 GitHub

- O projeto pode ser realizado em **Python (preferencial)** ou em **MATLAB**.
- O repositório GitHub deve conter código-fonte, documentação (**README.md**), prints da IHM e resultados.
- Cada integrante deve realizar commits significativos.
- Será valorizada a clareza da explicação matemática, organização do código e qualidade da interface gráfica.

5 Recursos Adicionais

5.1 Links Úteis

- Documentação PyQt5: <https://doc.qt.io/qtforpython/>
- Documentação PyQtGraph: <https://pyqtgraph.readthedocs.io/>
- Biblioteca Python Control: <https://python-control.readthedocs.io/>
- Documentação NumPy/SciPy: <https://numpy.org/doc/>

Referências

- [1] Smith, C. L., *Digital Computer Process Control*, Intext Educational Publishers, 1972.
 - [2] Sundaresan, K. R., Krishnaswamy, P. R., *Estimation of time delay parameters in linear systems*, Industrial & Engineering Chemistry Process Design and Development, 1978.
 - [3] Ziegler, J. G., Nichols, N. B., *Optimum settings for automatic controllers*, Transactions of the ASME, 1942.
 - [4] Rivera, D. E., Morari, M., Skogestad, S., *Internal Model Control: PID Controller Design*, Industrial & Engineering Chemistry Process Design and Development, 1986.
 - [5] Chien, K. L., Hrones, J. A., Reswick, J. B., *On the automatic control of generalized passive systems*, Transactions of the ASME, 1952.
- Ogata, K. (2010). *Engenharia de Controle Moderno*. 5^a Edição.
 - Åström, K. J., & Hägglund, T. (2006). *Controle PID Avançado*.
 - Seborg, D. E., Edgar, T. F., & Mellichamp, D. A. (2016). *Dinâmica e Controle de Processos*. 4^a Edição.
 - Dorf, R. C., & Bishop, R. H. (2016). *Sistemas de Controle Modernos*. 13^a Edição.